Task 2

Data-sheet parameters that constrain the design:

Motor supply voltage —> operates upto 46V, the wide limit ensures that there is no voltage worry on the motor side.

Voltage drop across the bridge(roughly 2V at about 1A, more at 2A) —> 11..1V on battery gives 9V at the motor (10.6V when full). That's why we chose 12V rated motors instead of 6V ones.

Condition: Exceeding the output current / junction temperature limit

The L298 is rated 2 A DC per channel with a maximum junction temperature of 150 °C.Because its bipolar outputs drop about 2 V, a motor drawing current I dissipates roughly 2 V × I as heat in the chip; at 2 A that is about 4 W per channel, which the small heat sink on the module cannot remove for long. A motor held in stall (wheel blocked at a wall or maze bump) is the realistic way to hit it.

How the design avoids it:

PWM is capped at 200/255 in firmware (MAX_SPEED).

Lost-line logic stops the motors after 0.3 s of searching instead of driving into a wall indefinitely
The module's heat sink is left open to airflow and the board is placed away from the battery.

Second, module-specific hazard: The red L298N module has an onboard 5 V regulator (78M05 type) enabled by a jumper cap. Module guides advise using it only when the motor supply is about 12 V or below; a full 3S pack is 12.6 V. This design therefore removes the jumper and feeds the module's 5 V terminal from the external MP1584 buck converter, so the regulator never overheats.

Task 3

PID decides how hard to steer:

- **P** (proportional) steers in proportion to the error. It's like turning the wheel harder the further you drift.
- **I** (integral) fixes a long-lasting small error.
- **D** (derivative) looks at how fast the error is changing and brakes the steering to stop wobbling.

- We will us PD configuration for our bot, as the error is entered on zero flips sign constantly so there is little steady state offset for an integral term to remove.

P gives the main steering and D supplies the damping that allows a higher cruising speed.

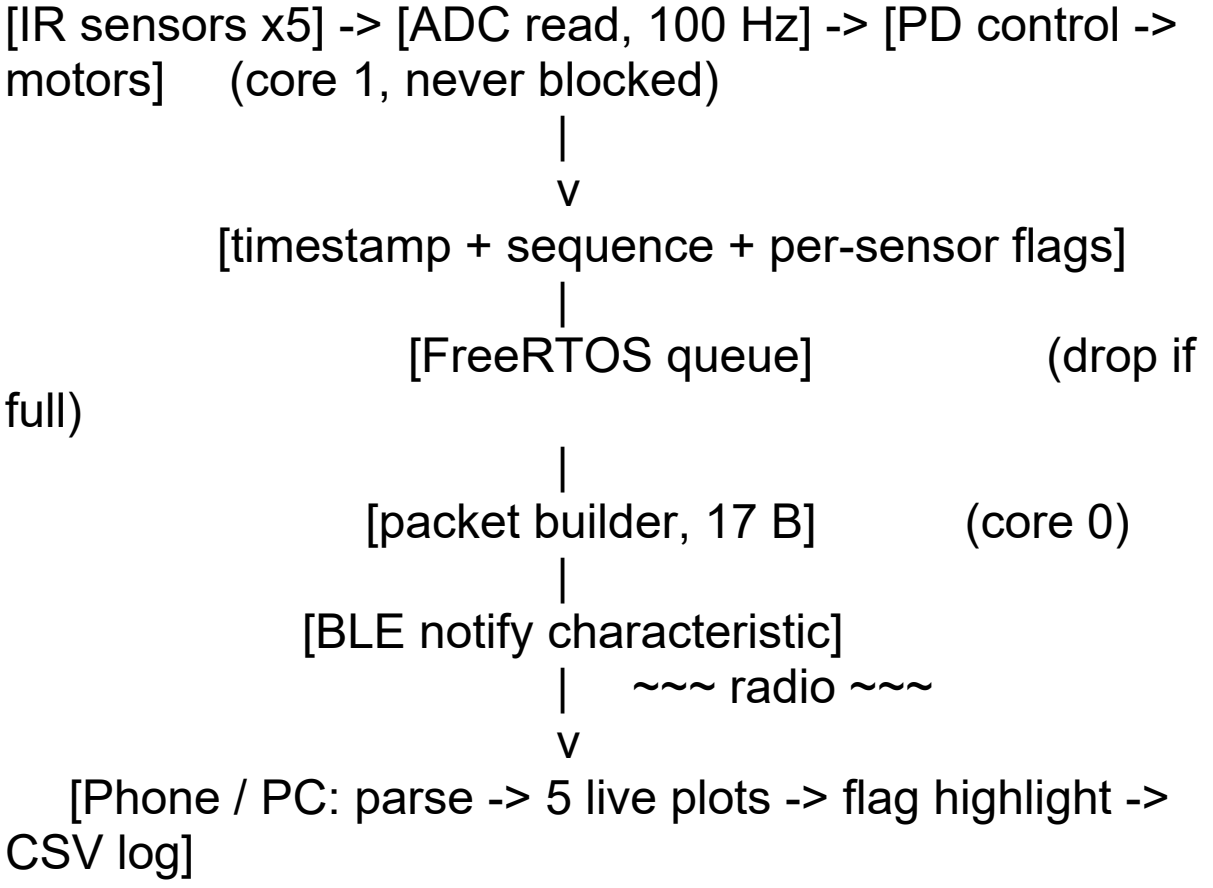
Sensor readings to motor commands:- First sensor readings are normalised then position is found out with weighted centroid, now the error is calculated for the correction step so that wheel left and right motor rotates correspondingly to the correction with each of them clamped to a max value.

Task 7

Protocol selection:-

We will use BLE with notifications over wifi solely because its disables ADC2 which would corrupt the sensor on GPIO25 pin. Also its ease of implementation is moderate compared to wifi, but latency is low and power load is the lowest.

Block diagram:-



Task 6

Troubleshooting: the ESP32 resets when the motors engage

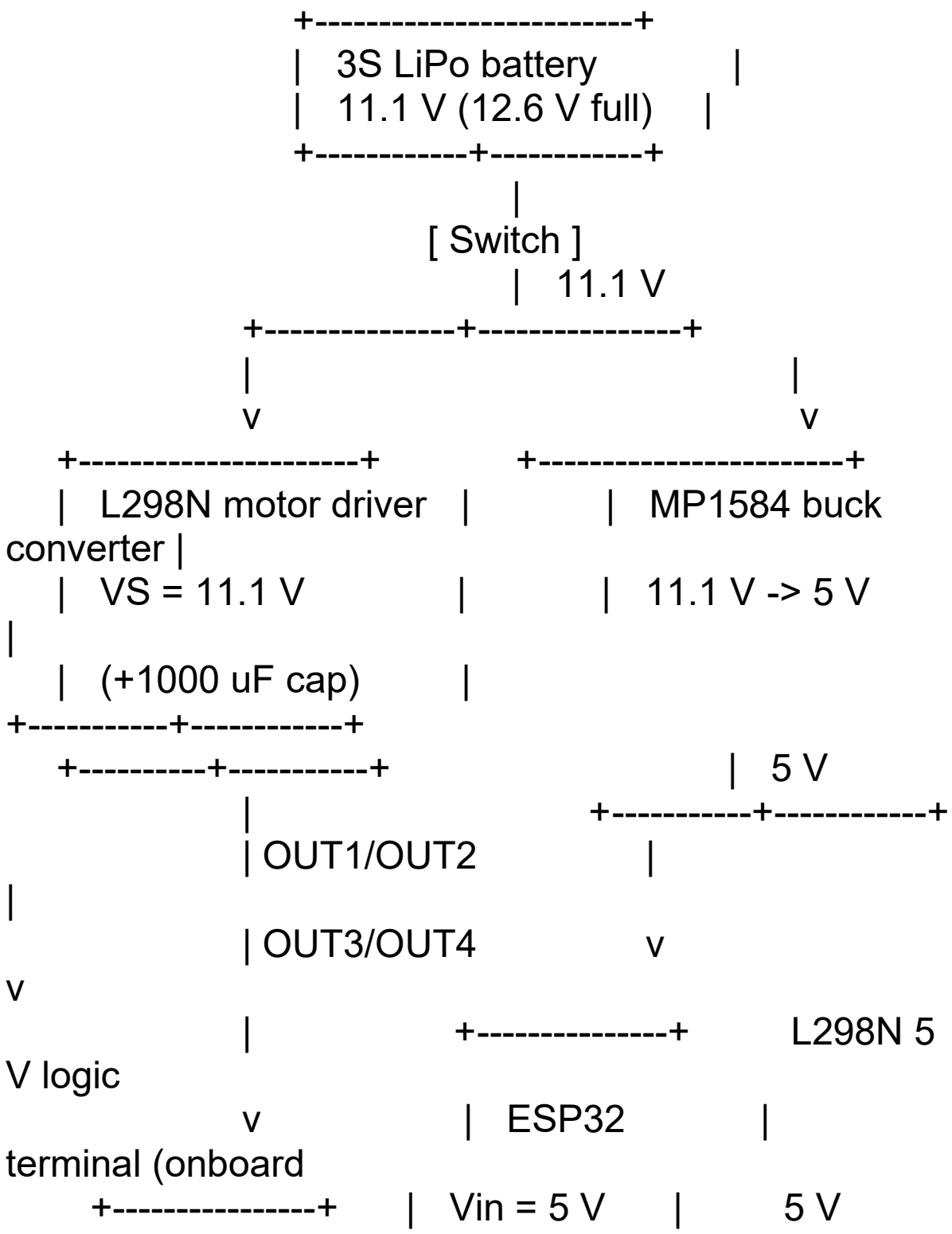
Probable root causes:-
• Supply dip / brownout—>Motor start-up and stall currents (up to ≈1.6 A for both) pull the battery and 5 V rail down for a few ms; the ESP32's 3.3 V falls and its brownout detector resets it.
• Insufficient bulk capacitance near the L298N and ESP32, or a weak/undersized 5 V source.
• Shared ground impedance—>thin or long ground wires shared by motor and logic return currents lift the ESP32's ground.
• Motor noise / inductive spikes coupling into the supply (no 0.1 µF across motors, flyback path missing).
• Battery or connector problem—> a poor XT60/screw-terminal joint or a tired cell with high internal resistance.

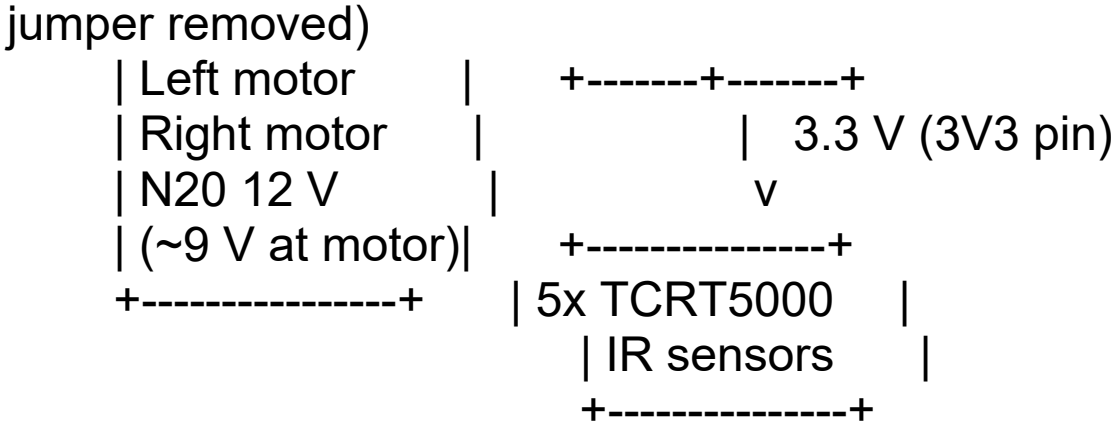
First diagnostic check—>Open the serial monitor at 115200 and read the reset reason printed at boot: a message such as "Brownout detector was triggered" confirms a supply dip rather than a code crash

Fix and confirmation:-
• Power the ESP32 from the separate buck (not the L298N regulator)
• Add 1000 µF across the L298N VS , 100 µF on the 5 V rail near the ESP32, and 0.1 µF ceramics across each motor's terminals.

• Add a software soft-start (ramp PWM up over about 100 ms) so the inrush current is spread out.

POWER BLOCK DIAGRAM (power only; signal wires not shown)





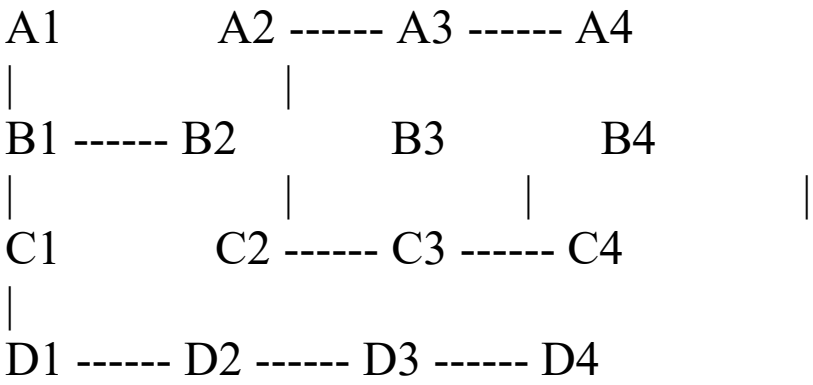
GROUND: battery -, buck, L298N GND, ESP32 GND, sensors and motor driver
all connect to ONE common ground rail.

RAIL SUMMARY
11.1 V : battery -> switch -> L298N VS and buck input
5 V : buck -> ESP32 Vin and L298N 5 V logic terminal
3.3 V : ESP32 3V3 pin -> IR sensors
~9 V : L298N output to motors (about 2 V lost inside the L298N)

Task 8

Choice for an unknown maze:
left-hand rule with turn recording. It needs no localisation, no extra hardware beyond the line sensors, and is reliable on the usual line-maze rule (a simply connected maze). Recording each decision lets the robot later cut out dead ends. But if the maze contains loops, we should switch to DFS.

Demonstration:



START = A1 (dead-end stub) GOAL = D4
Junctions (3 exits): B1, B2, C3
Dead ends: A4, B3, B4 (A1 = start, D4 = goal)
Corners / straights (no decision): A2, A3, C1, C2, C4, D1, D2, D3

OPTIMAL ROUTE (6 edges):
A1 -> B1 -> C1 -> D1 -> D2 -> D3 -> D4

LEFT-HAND-RULE TRAVERSAL (24 moves, priority Left > Straight > Right > Back):
A1 B1 B2 A2 A3 A4 A3 A2 B2 C2 C3 B3 C3 C4 B4 C4 C3 C2 B2 B1 C1 D1 D2 D3 D4

RECORDED DECISIONS (junctions and dead ends only):

B1:L B2:L A4:B B2:S C3:L B3:B C3:L B4:B C3:S B2:L B1:L

SIMPLIFICATION (angle sum mod 360: L=270, S=0, R=90, B=180):

L L B S L B L B S L L	<- LBS -> R
L R L B L B S L L	<- LBL -> S
L R S B S L L	<- SBS -> B
L R B L L	<- RBL -> B
L B L	<- LBL -> S
S	(final)

REPLAY: at junction B1 go straight, then follow the line to the goal.

Exploration vs optimal-route replay

• Exploration (unknown maze):
the robot knows nothing, so it must sense every junction, choose by rule, and handle dead ends and U-turns. It is slow and cautious and records its decisions.
• Replay (known route):
the shortened record is simply a list of the decision at each junction on the shortest path, here [S]. The robot follows the line and, at each junction, executes the next stored move. There is no search, no dead-end handling, no U-turns, so the logic is one lookup per junction and the robot can run faster.

Pseudocode

```
EXPLORE (left-hand rule, records decisions)
    record = []

    loop:
        follow line until a node is detected     (junction, dead end or goal)

        if goal marker detected:
            stop; break

        if left exit exists:            move = L
        else if straight exists:        move = S
        else if right exit exists:      move = R
        else:                           move = B        // dead end: U-turn

        if node is a junction or dead end:
```

```
            record.append(move)          // corners and straights are
not recorded

        perform the turn for move

        // simplify as we go: remove dead ends
        while len(record) >= 3 and record[-2] == B:
            a = record[-3]
            b = record[-2]
            c = record[-1]
            record[-3:] = [ combine(a, b, c) ]


combine(a, b, c)
    angle = (angle(a) + angle(b) + angle(c)) mod 360
    // angle(L)=270, angle(S)=0, angle(R)=90, angle(B)=180
    return move whose angle equals the result


REPLAY (known route)
    i = 0

    loop:
        follow line until a junction or goal is detected

        if goal marker detected:
            stop; break

        perform the turn for record[i]
        i = i + 1
```

Task 9 - LMS Implementation Challenge

ESP32 line follower, PD control. Files: lf_core.h, line_follower.ino, tests/test_core.cpp.

Approach

Logic is in lf_core.h (no hardware calls, so it can be tested on a PC); line_follower.ino only handles pins, calibration, the 10 ms loop, motors and serial output. Each loop: normalise sensors, find the weighted centroid, error = 0 - position, PD correction, left = base - corr, right = base + corr, clamp, drive. If the line is lost, spin toward the last side for 0.3 s, then stop.

Changes to Appendix A

- IN4 moved from GPIO 11 (flash pin, unusable) to GPIO 4.
- GPIO 25 is ADC2, not ADC1; fine with Wi-Fi off.
- GPIO 12 is a strapping pin; kept per spec, remap if boot fails.
- analogWrite needs Arduino-ESP32 3.x; LEDC fallback for 2.x.
- Loop is 10 ms because 115200 baud cannot carry a telemetry line every 5 ms.

Test run (sample sensor arrays)

```
Steady-state samples (D = 0):
case                pos   error     corr     L     R
centred           0.000   0.000     0.00   140   140
slightly right    0.333  -0.333   -10.00   150   130
far right         1.653  -1.653   -49.59   190    90
slightly left    -0.333   0.333    10.00   130   150
far left         -1.653   1.653    49.59    90   190
all white          LOST       -        -     -     -

Lost-line sequence (line last seen on the right):
  lost for   1 loops -> L= 110 R=-110
  lost for  15 loops -> L= 110 R=-110
  lost for  30 loops -> L= 110 R=-110
  lost for  31 loops -> L=   0 R=   0
  lost for 100 loops -> L=   0 R=   0

D term: first sample corr=-90.0, steady sample corr=-30.0

All assertions passed.
```

Line at +0.333 gives corr -10: left 150, right 130 (turns toward the line). All white is detected as lost, then spin for 30 loops, then stop.

Tuning notes

- PD only: error changes sign constantly, so an integral term would wind up.
- Start: Kp 30, Kd 60, base 140, cap 200, loop 10 ms.
- Method: Kd 0, raise Kp until it weaves, back off to about 60 %, add Kd to damp, then raise speed.
- If it steers away from the line, swap the motor wires.
- Not tested on hardware; gains are starting values.

lf_core.h

```
// lf_core.h - hardware-independent line-following logic.
// Plain C++ so it runs on the ESP32 AND on a PC (see tests/test_core.cpp).
#pragma once
#include <math.h>

#define NSENS 5
static const float WEIGHTS[NSENS] = {-2, -1, 0, 1, 2};   // sensor positions

// ---- tunable constants (see notes in the PDF / README) ----
static const float KP          = 30.0f;  // PWM counts per unit of error
static const float KD          = 60.0f;  // PWM counts per unit change of error per loop
static const int   BASE_SPEED  = 140;    // cruise PWM (0..255)
static const int   MAX_SPEED   = 200;    // hard cap on any motor command
static const float LOST_SUM    = 250.0f; // sum of normalised readings below this = line lost
static const int   SEARCH_LOOPS= 30;     // 30 loops x 10 ms = 0.3 s of recovery spin
static const int   SEARCH_PWM  = 110;    // spin speed while searching

// Scale a raw ADC value to 0..1000 using its calibrated min/max.
inline float normalise(int raw, int mn, int mx) {
  if (mx <= mn) return 0.0f;                 // uncalibrated channel -> treat as white
  float v = (float)(raw - mn) * 1000.0f / (float)(mx - mn);
  if (v < 0) v = 0;
  if (v > 1000) v = 1000;
  return v;
}

// Weighted centroid. Returns false (and leaves pos alone) when the line is lost.
inline bool linePosition(const float v[NSENS], float &pos) {
  float sum = 0, wsum = 0;
  for (int i = 0; i < NSENS; i++) { sum += v[i]; wsum += WEIGHTS[i] * v[i]; }
  if (sum < LOST_SUM) return false;          // all-zero / too faint: no division
  pos = wsum / sum;
  return true;
}

inline int clampInt(int v, int lo, int hi) { return v < lo ? lo : (v > hi ? hi : v); }

// PD controller state.
struct PD { float prevErr; };

// error = target(0) - position. Returns the steering correction.
inline float pdUpdate(PD &c, float error) {
  float d = error - c.prevErr;               // change per loop (fixed loop period)
  c.prevErr = error;
  return KP * error + KD * d;
}

// Differential mix: positive correction slows the left wheel, speeds the right.
inline void mixMotors(float corr, int &left, int &right) {
  left  = clampInt((int)lroundf(BASE_SPEED - corr), -MAX_SPEED, MAX_SPEED);
  right = clampInt((int)lroundf(BASE_SPEED + corr), -MAX_SPEED, MAX_SPEED);
}

// Lost-line recovery: spin toward the side where the line was last seen,
// then stop safely if it is still not found.
inline void recovery(float lastPos, int lostLoops, int &left, int &right) {
  if (lostLoops > SEARCH_LOOPS) { left = 0; right = 0; return; }   // safe stop
  if (lastPos >= 0) { left =  SEARCH_PWM; right = -SEARCH_PWM; }   // line was right
  else              { left = -SEARCH_PWM; right =  SEARCH_PWM; }   // line was left
}
```

line_follower.ino

```
// line_follower.ino - ESP32 (Arduino framework) line follower, PD control.
// Logic lives in lf_core.h (tested on a PC). This file only talks to hardware.
#include "lf_core.h"

// ---- IR sensors (Appendix A). NOTE: GPIO25 is really ADC2_CH8, not ADC1. ----
const int SENSOR_PIN[NSENS] = {34, 35, 32, 33, 25};

// ---- L298N pins. IN4 moved from GPIO11 (wired to flash, unusable) to GPIO4. ----
const int ENA = 14, IN1 = 27, IN2 = 26;   // left motor
const int ENB = 13, IN3 = 12, IN4 = 4;    // right motor (GPIO12 is a strapping pin)

const unsigned long LOOP_US = 10000;      // 10 ms = 100 Hz; keeps 115200-baud logging in budget

int calMin[NSENS], calMax[NSENS];
PD  pd = {0};
float lastPos = 0;                        // last known line position
int   lostLoops = 0;
unsigned long nextTick;

// analogWrite exists in Arduino-ESP32 3.x; 2.x needs the LEDC API.
#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  void pwmSetup(int pin, int)      { pinMode(pin, OUTPUT); }
  void pwmWrite(int pin, int, int v) { analogWrite(pin, v); }
#else
  void pwmSetup(int pin, int ch)   { ledcSetup(ch, 20000, 8); ledcAttachPin(pin, ch); }
  void pwmWrite(int, int ch, int v){ ledcWrite(ch, v); }
#endif

// cmd > 0 forward, < 0 reverse (swap IN pins if a motor runs backwards).
void setMotor(int in1, int in2, int en, int ch, int cmd) {
  if (cmd >= 0) { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); }
  else          { digitalWrite(in1, LOW);  digitalWrite(in2, HIGH); cmd = -cmd; }
  pwmWrite(en, ch, clampInt(cmd, 0, MAX_SPEED));
}

void drive(int left, int right) {
  setMotor(IN1, IN2, ENA, 0, left);
  setMotor(IN3, IN4, ENB, 1, right);
}

// Min-max calibration: sweep the robot over line and floor for 3 s at start-up.
void calibrate() {
  for (int i = 0; i < NSENS; i++) { calMin[i] = 4095; calMax[i] = 0; }
  unsigned long t0 = millis();
  while (millis() - t0 < 3000) {
    for (int i = 0; i < NSENS; i++) {
      int v = analogRead(SENSOR_PIN[i]);
      if (v < calMin[i]) calMin[i] = v;
      if (v > calMax[i]) calMax[i] = v;
    }
    delay(2);
  }
}

void setup() {
  Serial.begin(115200);
  int outs[] = {IN1, IN2, IN3, IN4};
  for (int k = 0; k < 4; k++) pinMode(outs[k], OUTPUT);
  pwmSetup(ENA, 0); pwmSetup(ENB, 1);
  drive(0, 0);                        // motors off while calibrating
  analogReadResolution(12);
  Serial.println("Calibrating: sweep sensors over line and floor...");
  calibrate();
  nextTick = micros();
}

void loop() {
  // fixed-period loop so the D term is a consistent "change per 10 ms"
  while ((long)(micros() - nextTick) < 0) { }
  nextTick += LOOP_US;

  float v[NSENS];
  for (int i = 0; i < NSENS; i++)
    v[i] = normalise(analogRead(SENSOR_PIN[i]), calMin[i], calMax[i]);

  int left, right;
  float pos = 0, err = 0, corr = 0;
  if (linePosition(v, pos)) {
    lostLoops = 0;
    lastPos = pos;
    err  = 0 - pos;
    corr = pdUpdate(pd, err);
    mixMotors(corr, left, right);
  } else {
    lostLoops++;
    pd.prevErr = 0;                   // avoid a D kick when the line returns
    recovery(lastPos, lostLoops, left, right);
  }
```

```
  drive(left, right);

  // telemetry every loop: sensors, position, error, correction, motors
  for (int i = 0; i < NSENS; i++) { Serial.print((int)v[i]); Serial.print(','); }
  Serial.printf("%.2f,%.2f,%.1f,%d,%d\n", pos, err, corr, left, right);
}
```